Week 9 Write Up for

Master of Science

Information and Communications Technology

Melissa Knapp

University of Denver University College

May 31, 2026

Faculty: Nathan Braun, MA

Director: Cathie Wilson, MS

Dean: Michael J. McGuire, MLS

Table of Contents

Design Overview

The primary challenge pertains to tracking visitors across various parking facilities and invoicing them monthly according to the duration of their parking. The system encompasses functionality for registering customers and their vehicles, as well as documenting parking locations and time entries. This week, we implemented multithreading in the codebase using different techniques like extending Thread or implementing the Runnable interface. We also researched thread pools to prevent constant new thread creation during the programs runtime.

I decided to implement the Runnable interface in my multithreading design. The main reason for this is something I read in my research which was that if you don't need any of the other methods the Thread class has, it's simpler to implement Runnable. And as I didn't need access to any methods besides run(), I stuck with the Runnable Interface. As for the thread pool portion of my design, I went with Executor and ExecutorService to create a finite thread pool to use for client connections. From my research I couldn't find any great benefits to using another option and Executor seemed the simplest to implement from what I was reading.

Challenges Faced

At first, I was very overwhelmed by the thought of refactoring the codebase to use multithreading. But making small changes at a time from moving client handler logic to its own function, to its own class, to implementing runnable made it much more manageable. The biggest problem I had was I initially was passing the client.getOutputStream() and client.getInputStream() to the Runnable class that I had created and was getting an error message that my input was null. So, I ended up passing the socket itself into the Runnable class and that fixed the error.

Threading Models

When working with threading designs I tried both manually creating threads and thread pools using ExecutorService. I didn't see much difference in terms of the amount of time taken between the two. From what I understand the ExecutorService will have less overhead in terms of CPU usage, but it likely isn't noticeable until you have many more connections coming in. Even so, thread pools still provide an important structural advantage because they reuse a limited number of threads instead of continuously creating new ones. That makes the application more efficient under heavier workloads and helps prevent resource exhaustion as the number of client requests grows. While the performance gains were minimal in my small test case, the design benefits of ExecutorService make it a more practical option for a production-style environment.

Reflection

In terms of the changes made, the script I used to test timings initially had only two commands, so the differences I observed were not very significant. The one larger change—moving the client handler logic into its own method—may have been a temporary anomaly rather than evidence of a slower implementation. Because the test included so few commands, outliers had a greater effect on the overall completion time. To improve the accuracy of my results, I added eight more commands and was then able to see clearer timing differences when comparing the progression from method, to class, to *Runnable*, and finally to thread pools. This gave me a better understanding of how each design choice affected performance and reinforced the importance of running broader tests before drawing conclusions. Overall, the additional

testing showed that thread pools offered the most stable and scalable approach for handling multiple client connections.